

Experiment 3:

Working with Gradle: Setting Up a Gradle Project, Understanding Build Scripts (Groovy and Kotlin DSL), Dependency Management and Task Automation.

Step 1: Setting Up Gradle on Ubuntu

Step 1: Update the System

```
sudo apt update
```

Step 2: Install Java

```
sudo apt install openjdk-17-jdk
```

Check the installed Java version:

```
java -version
```

Step 3: Install gradle

```
sudo apt install gradle -y
```

Check the installed gradle version:

```
gradle -v
```

Step 2: Steps to Create a Gradle Project

Step 1: Create a new directory for your project and navigate to new directory created.

```
mkdir my-gradle-project
```

```
cd my-gradle-project
```

Step 2: Initialize a Gradle project

```
gradle init
```

Select type of project to generate:

1: basic

2: application

3: library

4: Gradle plugin

Enter selection (default: basic) [1..4] 2

Select implement language:

1:Kotlin

2:Groovy

Enter selection (default: Kotlin) [1..2] 2

Open build.gradle and add

plugins {

id 'application' // Apply the application plugin

}

Add the following at the end in build.gradle

application {

mainClassName 'com.example.App' // Replace with your main class

path

}

Step 3: Understanding Build Scripts (Groovy and Kotlin DSL)

The build.gradle file you created uses Groovy DSL by default. And also supports Kotlin DSL.

build.gradle (Groovy DSL)

```
plugins {  
    id 'java'    // Apply the Java plugin  
}  
repositories {  
    mavenCentral() // Use Maven Central to download dependencies  
}  
dependencies {  
    testImplementation 'junit:junit:4.13.2'    // Test dependencies  
}
```

build.gradle.kts (Kotlin DSL)

```
plugins {  
    java // Apply the Java plugin  
}  
repositories {  
    mavenCentral() //Use Maven Central to download  
dependencies  
}  
dependencies {  
    testImplementation("junit:junit:4.13.2") // Test dependencies  
}
```

Step 4: Dependency Management with Gradle

In Gradle, dependencies are libraries or packages your project needs in order to run (for example, JUnit for testing, Spring for web development).

Adding Dependencies:
Open the build.gradle file.
Add dependencies in the dependencies block.

```
dependencies {  
    implementation 'org.springframework:spring-core:5.3.10' // Core  
    Spring framework  
    testImplementation 'junit:junit:4.13.2' // JUnit for testing  
}
```

Gradle will automatically download these dependencies from Maven Central and store them in your local cache.

Run Gradle Commands

```
gradle build  
gradle test  
gradle run
```

Step 5: Task Automation in Gradle

We can define your own tasks to automate specific actions.
In your build.gradle file, add a custom task:

```
task hello  
{ doLast {  
    println 'Hello, Gradle!'  
}  
}
```

```
build.gradle.kts  
tasks.register("hello")  
    { doLast {  
        println("Hello, Gradle!")  
    }  
}
```

Step 6: Run this on your terminal

```
gradle hello
```